

Student Expense & Budget Manager

PROJECT REPORT

VITyarthi - Build Your Own Project
CSE1021 | Java Course Project
VIT Bhopal University

Submitted by

Aryan Rai

Registration No.: 25BAI10726

Java command-line application

Expense management | Budget management | Reporting

Technical basis: final uploaded README (4).md

Report date: 17 September 2026

1. Introduction and project scope

Student Expense & Budget Manager is an offline Java command-line application for recording daily expenses, managing monthly budgets and reviewing spending. It uses the Java Standard Library and stores information in local files, allowing the application to run without a database, an IDE or a network connection. [1]

Problem statement

Small purchases are easy to forget. Students need a simple way to understand where their money goes and whether their spending exceeds a monthly budget. This project brings expense records, monthly limits and spending summaries into one menu-driven application. [1]

Objectives

The project aims to record and retrieve daily expenses; organize spending by category and month; compare expenditure with a monthly budget; identify overspending; and preserve records between sessions. It also demonstrates Java object-oriented programming, collections, validation, date handling and file persistence in a practical application. [1]

Scope and evidence basis

The target user is a student operating a local CLI. The documented scope includes adding, viewing, searching and deleting expenses, setting and viewing monthly budgets, and producing monthly reports. Editing expenses is a future enhancement. This report describes the final README; source code, the referenced verification files and live test output were not supplied as report evidence. [1]

Diagrams are conceptual views reconstructed from documented responsibilities. They do not assert source-level method signatures, exact internal call order or undocumented file syntax. The results section uses only the README example and identifies it as illustrative.

Report guide

Requirements - 3; System architecture - 4; Use cases and workflow - 5; Sequence design - 6; Components and storage - 7; Design and implementation - 8; Command-line setup - 9; Testing and results - 10; Limitations and learnings - 11; Repository and references - 12.

2. Requirements

2.1 Functional requirements - three major modules

Module	Inputs and required behavior	Outputs
Expense management	Store ID, date, category, description and amount. Add, view, search and delete expenses. Validate entered values.	Expense records, matching records, deletion outcome and input errors.
Budget management	Set and view a separate budget for each month. Calculate spending and remaining balance.	Monthly budget, spending, remaining balance and overspending warning.
Reporting	Select a month and aggregate its expense records.	Monthly total, category totals, expense count, highest/lowest expense and remaining budget.

2.2 Non-functional requirements

Requirement	Documented expectation
Usability	Numbered menu, named categories, clear formats and error messages.
Reliability	Validate before saving; recover from invalid input; preserve malformed source files for repair.
Maintainability	Seven production classes separate CLI, business logic, storage and validation responsibilities.
Performance	Linear searches and totals for ordinary student records. Each mutation rewrites its file; no large-scale performance claim is made.
Persistence	UTF-8 files are saved after each successful change. Temporary-file replacement reduces partial-write risk.
Privacy	Offline local storage; personal data files are excluded from Git. Files are not encrypted.

Source: README, Functional Requirements and Non-Functional Requirements. These three modules and six quality requirements address the corresponding minimum counts in the VITyarthi guidelines. [1, 2]

3. System architecture

The application separates terminal interaction, business responsibilities and local persistence. Main provides the menu; the three service classes represent expense, budget and reporting operations. Expense holds the expense model, while FileManager and InputValidator provide storage and validation responsibilities. [1]

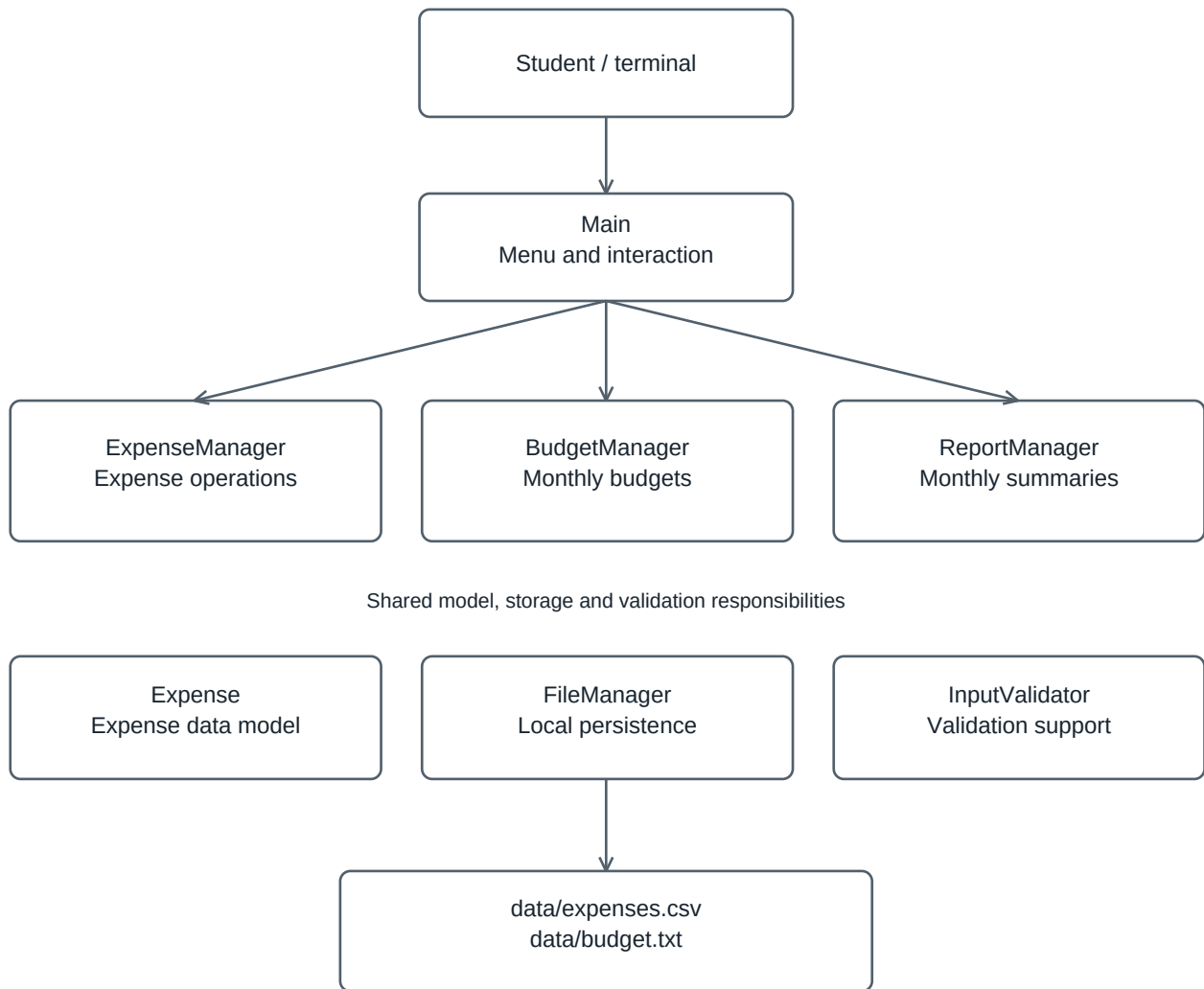


Figure 1. Conceptual architecture based on the README class structure. Connections express logical responsibility, not verified source-level dependencies.

Input, processing and output

A student supplies an operation and its data at the terminal. Input is checked before a change is saved. Expense and budget operations maintain the information used by the monthly reporting module. Output appears in the CLI as records, summaries, balances, warnings or errors. Successful changes persist locally. [1]

Deployment boundary

The program runs on a machine with JDK 11 or newer. Runtime data is kept in the selected local data directory. Only one application instance should use a given data directory at a time. There is no external runtime dependency or database requirement. [1]

4. Use cases and workflow

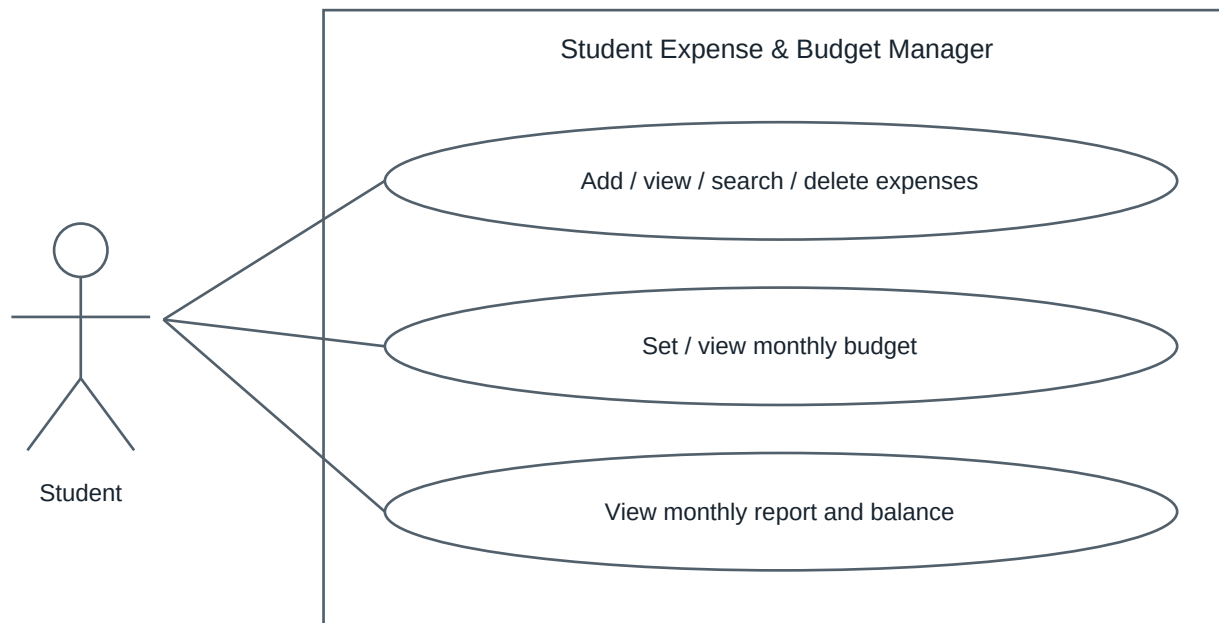
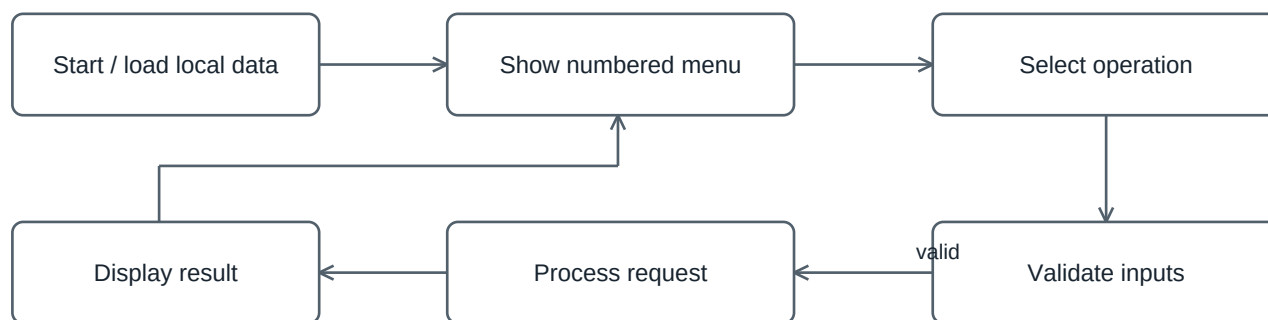


Figure 2. Use case diagram. One student actor accesses the three documented functional modules.

Expense operations use the stored ID, date, category, description and amount. Budget operations select a month and its limit. Reporting selects a month to summarize recorded spending and its relationship to the budget. [1]



Successful changes are saved locally. Invalid input returns to the menu.

Exit option (8) or end-of-input ends the session.

Figure 3. Conceptual menu workflow. Validation failure cancels the operation and returns to the menu; successful changes are persisted. [1]

Interaction rules

Search is case insensitive across descriptions and categories, substring-based for dates, and exact for ID text. Matching any of these fields includes a result. Listing preserves insertion order. Deletion takes a numeric ID. A blank month selects the current system month. Invalid entry cancels the operation; the user selects it again to retry. End-of-input exits cleanly. [1]

5. Sequence design

Adding an expense - successful path

The sequence below explains the logical collaboration needed to add a valid expense and persist a successful change. Message labels describe responsibilities rather than Java method names. The README does not specify the exact order of internal calls or object updates. [1]

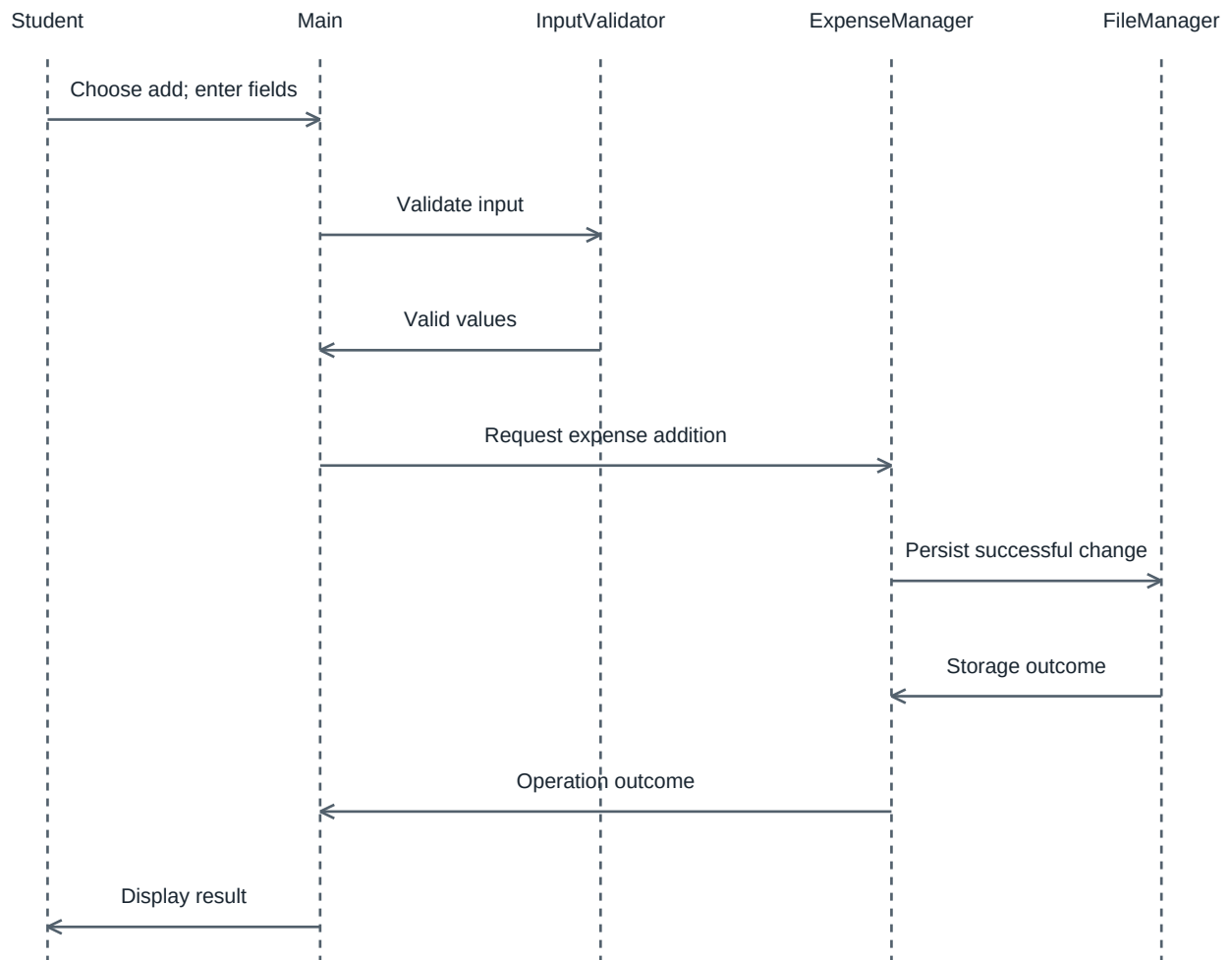


Figure 4. Conceptual sequence for a successful expense addition, derived from the documented CLI, validation, expense and storage responsibilities.

Alternative outcomes

Invalid input: the operation is cancelled and control returns to the menu. The student can select the operation again to retry. Validation occurs before saving. [1]

Storage errors: exception handling covers I/O errors. The documented test coverage includes failed writes and malformed files; malformed source files are preserved for repair. No specific recovery algorithm or error text is asserted here. [1]

6. Components and storage design

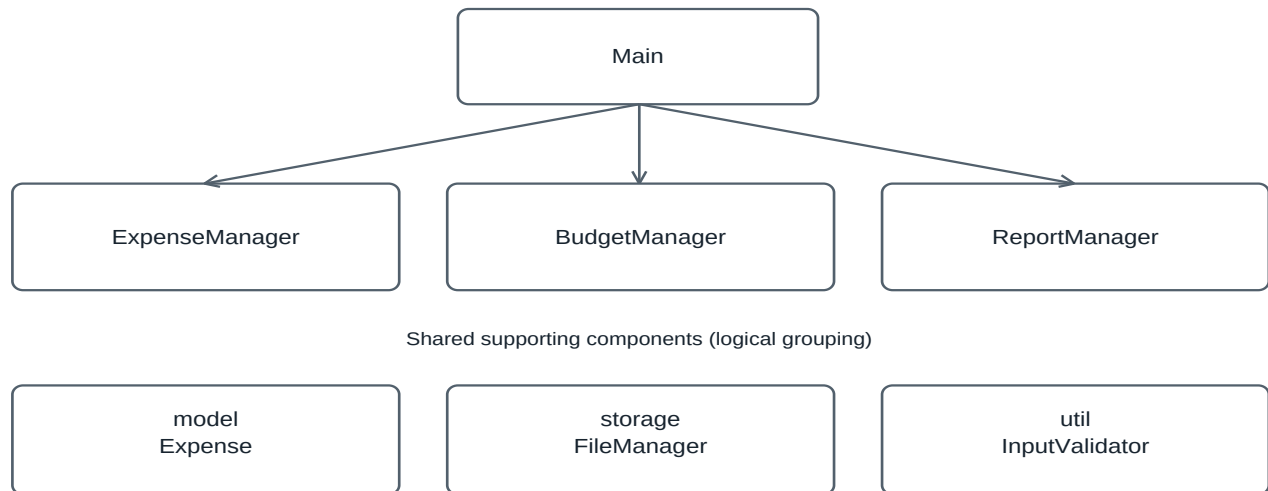
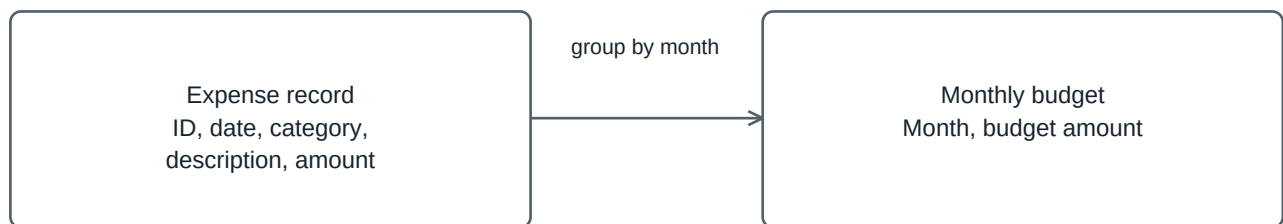


Figure 5. Component view of the seven production classes. Package labels follow the README. No undocumented inheritance or method signatures are assumed.

6.1 Logical storage model



Logical association through date / YearMonth; no database foreign key.

Figure 6. Logical entity association for file-backed storage. Expenses are selected by month to compare spending with a monthly budget; this is not a relational database schema.

Local file	Logical contents and purpose
data/expenses.csv	Expense records: ID, date, category, description and amount. Supports retrieval and monthly aggregation.
data/budget.txt	Separate monthly budgets, logically associating a month with its budget amount.

Both files are generated on first execution and excluded from Git. Files use UTF-8 and are saved after each successful change, with temporary-file replacement reducing partial-write risk. A different data directory can be selected at launch. [1]

The README does not define column order, CSV escaping, headers, budget delimiters, ID generation rules or exact serialization. Accordingly, this report specifies the logical data model without inventing an on-disk schema. BigDecimal handles currency; LocalDate and YearMonth support dates and month selection. [1]

7. Design rationale and implementation

Documented choice	Rationale and trade-off
CLI and Java Standard Library	Keeps execution accessible from a terminal without additional runtime dependencies. The interface is text-based.
Separate production classes	Separates interaction, business logic, persistence and validation, making responsibilities easier to understand and maintain.
BigDecimal for currency	Supports exact decimal money arithmetic, which is important when adding expenses and subtracting them from a budget.
LocalDate and YearMonth	Represents dates and months directly for validation and month-based selection.
Local UTF-8 files	Preserves records across runs without a database. Each mutation rewrites a file; the README makes no large-scale performance claim.
Temporary-file replacement	Reduces partial-write risk. It is not a stated guarantee of crash-proof or concurrent storage.

Rationale above explains the documented choices; it is not a record of the developer's personal decision-making process. [1]

Java concepts and tools

Expense and the managers demonstrate classes, objects and constructors. Private fields and getters demonstrate encapsulation. ArrayList, maps, sets and comparison operations support collection processing. Small lambdas support searching, deletion and comparisons. Packages organize model, service, storage and util responsibilities. [1]

Files and Path support file handling; exceptions handle validation and I/O errors. The CLI uses Scanner, loops, conditionals and methods. Tools listed in the README are the JDK command-line tools, Git, Markdown and Mermaid diagrams. [1]

Implementation organization

The src folder contains Main.java and the model, service, storage and util packages. tests/TestRunner.java supplies the test entry point. README.md and statement.md document the project. The docs folder lists architecture.md, diagrams.md, study-guide.md, submission.md, verification.md and cli-session.txt. The data folder includes .gitkeep; personal runtime data is excluded from version control. [1]

8. Command-line setup and execution

Install JDK 11 or newer; a JRE alone is insufficient. Ensure both java and javac are available on PATH. The README states that the commands work in PowerShell, Windows Command Prompt and ordinary Unix shells. Run them from the project root. [1]

1. Check the environment

```
java -version
javac -version
```

2. Obtain the project

Download and extract the project, then open a terminal in its root folder. If the project has been published to the intended account, the README gives this alternative:

```
git clone https://github.com/sancharika25BAI10726-creator/student-expense-budget-manager.git
cd student-expense-budget-manager
```

3. Compile the application - exact README command

```
javac -d out src/model/*.java src/service/*.java src/storage/*.java src/util/*.java src/Main.java
```

4. Run the application

```
java -cp out Main
```

Keep the working directory at the project root to reuse the same data directory. To select another data folder, use:

```
java -cp out Main demo-data
```

Use only one running application per data directory. No additional dependency installation or configuration is required beyond the JDK. [1]

5. Compile and run the tests after compiling the application

```
javac -cp out -d out tests/TestRunner.java
java -cp out TestRunner
```

The test runner creates an isolated temporary directory and deletes only its own test data. A failed check throws an error and causes an unsuccessful exit. The commands are reproduced from the README; this report does not claim they were executed. [1]

9. Testing approach and results

9.1 Documented testing approach

tests/Runner.java provides the documented automated checks. The coverage below summarizes the README description, not independently observed pass results. The README refers to docs/verification.md for actual execution evidence and menu checks; that file was not included in the supplied report sources. [1]

Area	Coverage stated by the README
Expense behavior	Expense operations and month filtering.
Money and budgets	Exact money arithmetic and budgets.
Reporting	Report calculations and summaries.
Validation	Input validation.
Persistence and failures	Persistence, malformed files and failed writes.

9.2 Supported illustrative result

The README provides a single-record scenario. It is an illustration of expected behavior, not a captured run or a fabricated terminal transcript. [1]

Step	README example
Set budget	Choose 5; enter 2026-09 and 1000.
Add expense	Choose 1; enter 2026-09-17, Food, Lunch and 125.50.
View report	Choose 7; enter 2026-09.
Remaining budget	Rs. 1000.00 - Rs. 125.50 = Rs. 874.50.
Exit	Choose 8.

9.3 Result interpretation

The example demonstrates how a budget and one expense relate to the remaining balance. The README also documents category totals, count and highest/lowest expenses as report outputs. No screenshots, measured timings, test counts, pass rates or real execution output are asserted in this report. [1]

10. Limitations and key takeaways

10.1 Technical challenges and limitations

The following are design concerns reflected in the README, not invented personal experiences of development. [1]

Concern	Documented response or boundary
Money and date correctness	BigDecimal provides exact currency arithmetic; LocalDate and YearMonth validate dates and select months.
Invalid input and damaged files	Invalid input cancels the operation and returns to the menu. Malformed source files are preserved for repair.
Interrupted or failed writes	Temporary-file replacement reduces partial-write risk. Failed writes are included in the stated test coverage.
Data size and concurrent access	Linear searches and totals suit ordinary student records. Every mutation rewrites its file. One application should use each data directory.
Privacy and CSV scope	Records stay offline and are excluded from Git, but are not encrypted. Full CSV-library import/export is a future option if multiline fields become necessary.

10.2 Learnings and key takeaways

The documented design demonstrates how object-oriented decomposition separates a user interface from business and storage responsibilities. Collections support record processing, while dedicated money and date types make domain rules explicit. Validation and exception handling connect user input to reliable persistence. [1]

The test-runner design illustrates isolation of test data and coverage of error conditions as well as ordinary operations. Command-line instructions and a modular folder structure show how documentation helps another user build, run and evaluate a Java project. These are takeaways from the demonstrated concepts, not claims about personal learning experiences. [1]

10.3 Future enhancements

The README proposes editing existing expenses, exporting reports, recurring expenses, configurable categories, and import/export with a full CSV library if multiline fields become necessary. These are future possibilities and are not presented as implemented features. [1]

11. Repository information and references

11.1 References

[1] **Project README.** *Student Expense & Budget Manager*, uploaded as README (4).md. Primary source for all project-specific functionality, technologies, structure, commands, testing scope, limitations and future improvements.

[2] **VITyarthi.** *Build Your Own Project: General Project Instructions & Submission Guidelines*, uploaded as BuildYourOwnProjectVITyarthi.pdf. Source for functional and non-functional minimums, design artifacts, repository requirements and report sections.

[3] **VIT Bhopal University, Digital Outreach.** *VITyarthi: Evaluated Course Project - Flipped Course - CSE1021 - Reg*, email dated 11 September 2026, supplied as a PDF. Source for public visibility, root URL, README and command-line execution requirements.

11.3 Evidence boundary

References [1]-[3] are the supplied documents used to prepare this report. Files mentioned within the README, including source files and docs/verification.md, are described as documented artifacts and are not cited as independently inspected evidence. The conceptual diagrams and explanations remain bounded by the README; no additional implemented features or measured results are claimed.